

Interpreting Identity Safely

**A Technical Monograph on Tiny-IDP's Goja Microkernel, Explicit Continuations,
Invocation Capabilities, and Assurance-Oriented Runtime**

Source analysis prepared from TINYIDP-GOJA-001

2026-07-20

Table of Contents

Static contents for deterministic headless rendering; entries link to document headings.

Scope and source snapshot	4
Executive synthesis	5
Part I - The interpreter problem	7
1. Why identity scripting is unusually dangerous	7
2. Four stages, not one interpreter	7
3. The semantic firewall: pkg/idpprogram	8
Part II - Definition-time language and activation	9
4. A runtime type-and-effect system	9
5. Closed schemas and information-flow labels	9
6. The outcome algebra	9
7. Static workflow validation as finite-state analysis	10
8. The native module as a nominal type system	10
9. A closed CommonJS world	11
10. Deterministic callback registration	11
11. module.exports as an anti-ambiguity check	12
12. Immutable artifacts and defensive copies	12
Part III - Request-time Goja execution	13
13. Single-owner runtimes	13
14. The invocation context is copied, not shared	13
15. Invocation capabilities	13
16. The Promise bridge	14
17. Synchronous and asynchronous lambda results	15
18. Interrupts and the worker safety proof	15
19. Output decoding is a commit point	16
Part IV - Explicit durable continuations	17
20. A precise meaning of “serialized continuation”	17
21. Why VM serialization is rejected	17
22. The durable record as a typed security context	17
23. One-use transitions and linearizable replay control	18
24. Schema-checked carry and resume input	19
25. Generation pinning and semantic time travel	19
26. Cleanup is part of continuation semantics	19
Part V - Presentation, secrets, evidence, and native effects	20
27. Presentation is a typed protocol, not script-generated HTML	20
28. Presentation validation connects UI to the workflow graph	20
29. Exact POST projection	20
30. Request-scoped secret handles	21
31. commit as an algebraic effect request	21
32. Atomic signup commit	21
33. Native challenges and unforgeable evidence	22
34. Multi-request workflow execution	22
Part VI - Providers and policy interpreters	24
35. Providers are typed callback collections	24
36. Provider invocation reuses the same kernel	24
37. Authorization policy remains downstream of native proof	24
38. Claims are additive under native protection	24
39. Presentation policy is decoration-only	25
Part VII - Verification as a separate language	26
40. Production scripting and verification scripting are different interpreters	26
41. Step registries make scenario languages finite	26
42. Assurance vocabulary as an inter-interpreter ABI	26

Part VIII - A formal reading of the design	27
43. Defunctionalization and continuation-passing style	27
44. Algebraic effects without a general effect handler	27
45. Object capabilities and temporal authority	27
46. Staging and partial evaluation	28
47. Linearizability at the interpreter boundary	28
48. Nominal IDs as an ABI	28
Part IX - Security invariants	29
49. Core invariants and their enforcement points	29
50. Authority matrix	29
Part X - Critical evaluation	31
51. What is unusually strong	31
52. Residual risks and design tensions	31
53. Why the design is rigorous without pretending JavaScript is safe by itself	33
Part XI - Testing and operations as interpreter design	34
54. Tests target language-boundary failure modes	34
55. Shared store conformance	34
56. Embedded script tests and activation	34
57. Bounded observability	35
Part XII - Transferable engineering patterns	36
58. Pattern catalog	36
59. Review checklist for an embedded policy interpreter	37
Part XIII - Worked execution trace	38
60. Open signup from source to commit	38
Part XIV - Goja construct index	40
61. Goja APIs and their roles	40
62. Host-language constructs that complete the interpreter	40
Part XV - Source map	41
63. Recommended reading order	41
Conclusion	42

Scope and source snapshot

This report studies the `task/prod-tiny-idp` branch of `go-go-golems/tiny-idp` at commit

`d164ae59408bdd8bc21516274b446339b1761b1e`, dated 2026-07-20. The principal implementation and design sources are:

- the active lambda-first design, `design-doc/03-lambda-first-tiny-idp-javascript-api-with-explicit-browser-continuations.md`;
- the assurance-oriented synthesis, `design-doc/02-assurance-oriented-core-grammar-and-codebase-refactoring-proposal.md`;
- the implementation ledger and status record in `tasks.md` and `changelog.md`;
- the runtime-independent contract in `pkg/idpprogram`;
- the Goja compiler, runtime, capability bridge, and worker pool in `pkg/idpscript`;
- the native Goja module in `internal/gojamodules/tinyidp`;
- durable workflow state in `pkg/idpcontinuation`;
- typed browser projection and request-scoped secret handling in `pkg/idpworkflow`;
- the production signup integration in `pkg/idpsignup` and `internal/fositeadapter/scripted_signup.go`;
- typed policy/provider adapters, native challenge evidence, verification plans, and the assurance vocabulary.

The analysis is source-based. The repository's own ticket records extensive unit, race, conformance, and integration test runs. Those records are described where relevant, but this report does not present them as independently re-executed results.

The word *novel* is used here in the engineering sense: an unusual and rigorous synthesis within this codebase. It is not a claim of academic priority.

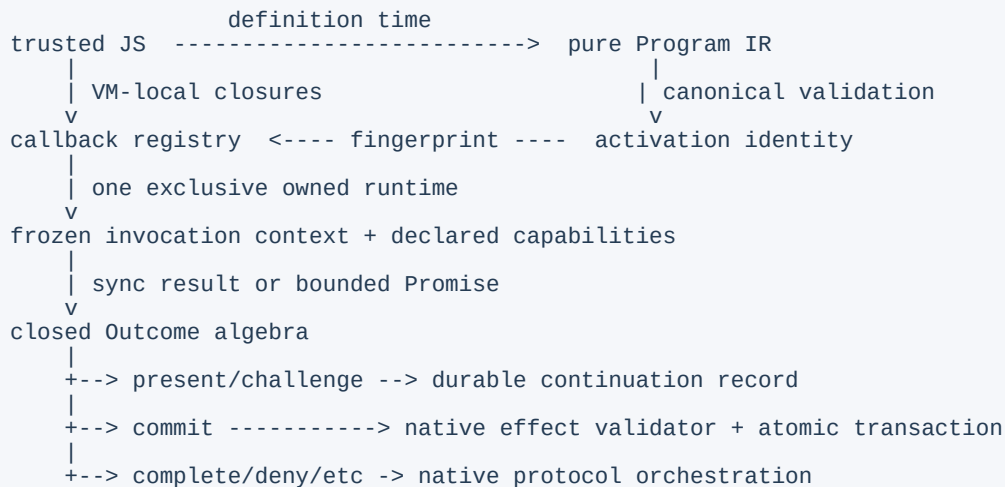
Executive synthesis

The significant achievement is not that Tiny-IDP embeds JavaScript. The significant achievement is that it refuses to make JavaScript the identity provider.

The branch implements a staged interpreter architecture in which JavaScript is allowed to author and execute narrowly typed policy fragments, while Go retains protocol authority, persistence authority, secret authority, HTTP authority, and atomic commit authority. The result is best understood as four related machines:

1. **A definition-time compiler.** Trusted JavaScript registers named lambdas and serializable workflow/provider contracts. It produces a pure-Go intermediate representation plus a VM-local callback registry.
2. **An activation verifier.** The same compiled source is materialized in independent owned runtimes. Program, schema, and callback fingerprints must agree before workers are accepted.
3. **A request-time lambda interpreter.** A worker receives a frozen, JSON-derived input object and only the capabilities declared by the selected lambda. It may perform bounded in-request asynchronous work and must return one member of a closed outcome algebra.
4. **A durable native workflow interpreter.** Browser-spanning control flow is not a suspended Promise or Goja heap. It is an explicit, versioned continuation record containing a handler label, typed public carry, exact generation identity, replay revision, and native references. Go validates and advances that record, renders pages, verifies challenges, and commits effects.

The architecture can be summarized as follows:



Several interpreter ideas reinforce each other:

- **Defunctionalization.** Durable continuations are represented by stable handler IDs and a schema-checked environment instead of serializing closures or stacks.
- **A runtime type-and-effect discipline.** Each lambda declares input and output schemas, allowed outcomes, required capabilities, permitted effects, timeout, call budget, and output budget.
- **Nominal branding in an untyped language.** Blank Goja objects are recognized by object identity in Go maps, making lambda, field, action, and secret handles unforgeable inside the VM.
- **Object-capability style authority.** A capability exists only because the host inserted it into one invocation, and it becomes inactive when the invocation ends.
- **Algebraic-effect style commits.** JavaScript returns inert effect plans. Native Go code validates the exact sequence and applies the effects in a named atomic operation.
- **Fail-stop worker leasing.** A runtime is returned to the pool only after a fully valid result and complete asynchronous settlement. Timeout, cancellation, exception, malformed output, or uncertain interruption causes disposal and replacement.
- **Generation-aware resumption.** A continuation is pinned to executable source plus semantic program identity. Hot reload creates a new generation rather than reinterpreting old state under new code.

- **Separate production and verification languages.** Production lambdas receive bounded policy capabilities. Verification JavaScript can only compile data-only scenarios, which are materialized against an explicit native step registry before execution.

This is a practical answer to a difficult question: how can an identity system gain expressive scripting without turning protocol correctness into a property of arbitrary script behavior?

Part I - The interpreter problem

1. Why identity scripting is unusually dangerous

A general embedded language normally seeks convenience: expose services, let scripts call them, and translate values. In an identity provider, that approach collapses several distinct authorities into one dynamic layer:

- protocol validation and OAuth/OIDC state transitions;
- browser request parsing, CSRF and origin handling, cookies, redirects, and response writing;
- password, one-time-code, invitation, and signing-key handling;
- transaction boundaries and one-time consumption;
- account, credential, session, consent, and token issuance;
- policy selection and presentation customization.

If all of those become ordinary methods on a script-visible host object, the scripting API becomes an alternate identity provider. Every script path must then preserve every protocol and storage invariant. Static review becomes difficult because authority is ambient, data types are open, and errors can accidentally change security meaning.

The active design takes the opposite route. JavaScript may choose among host-defined operations and return host-defined values, but it cannot own the transition that makes an identity assertion true. The design document explicitly excludes Fosite objects, stores, SQL transactions, keys, cookies, passwords, raw codes, and network clients from the script surface. See the design's authority table and core execution model in [design-doc/03, sections 4-6](#).

The central security move is therefore architectural rather than syntactic:

Script expressiveness is permitted only inside a native envelope whose inputs, outputs, capabilities, effects, time, size, lifetime, and continuation points are explicit.

2. Four stages, not one interpreter

It is tempting to describe the system as “Go executes JavaScript callbacks.” That misses most of the design. There are four different stages with different state and authority.

2.1 Stage A: definition-time evaluation

The top level of a source file executes in an isolated Goja runtime. It calls `require("tinyidp").v1`, constructs named lambdas, registers workflows/providers/tests, and exports the value returned by `A.program(...)`.

At this stage:

- Goja functions exist, but only inside the runtime's collector;
- the exported contract is data-only;
- no request, store, network, provider, or protocol object is present;
- source size and definition time are bounded;
- ambient module loading is disabled.

2.2 Stage B: activation materialization

A compiled `*goja.Program` is loaded into multiple independent owned runtimes. Each runtime reconstructs the program and callback registry. The host compares canonical program data and fingerprints. If one materialization differs, activation fails.

This stage is a form of reproducible linking: named callback references in the IR must resolve to the same VM-local registry in every worker.

2.3 Stage C: request-time invocation

For one handler invocation, the host:

- validates the JSON input against the handler's schema;
- leases one runtime exclusively;
- installs only the handler's declared capability bindings;
- creates request-scoped secret and evidence handles;

- builds and freezes a plain JavaScript context;
- invokes the named closure on the runtime owner;
- awaits a bounded Promise if returned;
- validates the resulting outcome;
- releases or destroys the worker.

2.4 Stage D: native transition interpretation

The JavaScript outcome is not the final protocol action. Go interprets it:

- `present` becomes a validated native page plus a durable continuation;
- `challenge` becomes a native challenge record and delivery operation;
- `commit` becomes a revalidated effect sequence and atomic transaction;
- `complete`, `deny`, `skip`, and `error` are normalized by the relevant native provider or protocol seam.

This fourth stage is what preserves the identity microkernel. The script proposes; the native transition commits.

3. The semantic firewall: `pkg/idpprogram`

The first hard boundary is a package with no Goja dependency. `pkg/idpprogram/program.go` defines a serializable `Program` containing workflows, providers, lambda specifications, schemas, capability descriptors, and tests. A lambda specification stores a callback ID, never a function.

This package is more than a DTO collection. It is the semantic firewall between two worlds:

Goja world	durable/native world
-----	-----
closures	callback IDs
objects with identity	stable string IDs
Promises	outcome records
host functions	capability requirements
mutable JS state	canonical JSON
runtime-local values	pure Go values

The separation creates three useful proof obligations:

1. **Closure locality:** no Goja value can enter a `Program` or continuation.
2. **Reference totality:** every callback ID in the IR must be present in each runtime's registry.
3. **Interpretation closure:** every possible handler result belongs to a finite native outcome family.

Part II - Definition-time language and activation

4. A runtime type-and-effect system

LambdaSpec assigns each callback a contract:

```
LambdaSpec =
  callback identity
+ lambda kind
+ input schema
+ output schema
+ allowed outcome set
+ required capability set
+ allowed effect set
+ timeout
+ maximum capability calls
+ maximum output bytes
```

This resembles a dynamic type-and-effect system.

- The input and output schemas are value types.
- AllowedOutcomes is a row of legal control effects.
- RequiredCapabilities is the authority context.
- AllowedEffects is the commit-effect context.
- Budget is a resource effect bound.

The contract is enforced twice. `idpprogram.Validate` checks it before activation. `pkg/idpscript/codec.go` checks actual inputs and outputs at invocation.

A useful way to read a handler declaration is:

```
under capability environment C,
within resource budget B,
this callback maps values of schema I
into one of outcome kinds O,
whose value conforms to schema R,
and whose commit plan may contain effects E.
```

That is far more reviewable than a generic `func(ctx map[string]any) any`.

5. Closed schemas and information-flow labels

`pkg/idpprogram/schema.go` defines a deliberately small schema language: object, string, boolean, integer, and bytes, with byte and length bounds. Object fields refer to named schemas. The validator rejects cycles and missing references.

Two design choices matter.

First, object schemas are closed by default. `Additional: false` makes unknown fields an error. This prevents a browser form, capability result, or script result from smuggling data into a handler that never declared it.

Second, fields carry a `Sensitive` marker. `ValidatePublicJSON` traverses a value under the destination schema and rejects sensitive fields. The same schema therefore serves two related purposes:

- structural typing for an ephemeral invocation; and
- an information-flow policy for durable public carry.

The continuation environment is not merely JSON that happens not to contain a password. It is JSON proven against a schema path that forbids sensitive fields.

6. The outcome algebra

`pkg/idpprogram/outcomes.go` defines a closed sum type:

- `continue` - immediate native dispatch to another handler;
- `present` - native UI plus a durable continuation;
- `challenge` - native proof mechanism plus a durable continuation;

- `commit` - inert effect plan for native validation and transaction;
- `complete` - terminal typed value;
- `deny` - valid negative policy result;
- `skip` - explicitly inapplicable provider/branch;
- `error` - infrastructure or internal failure.

The distinctions are security-relevant. A thrown exception is not a denial. `undefined` is not a skip. A rejected credential is not an invitation to silently try a weaker factor. The design document calls out this semantic discipline directly in [design-doc/03](#).

This is algebraic in the practical sense: the host performs exhaustive interpretation over a finite family. A new security-significant result requires a native enum, validation rule, interpreter branch, tests, and usually an effect or provider contract. It cannot arrive as an accidental JavaScript object shape.

7. Static workflow validation as finite-state analysis

A workflow is a map of handler IDs plus explicit continuation edges. The validator performs more than reference checking:

- handler and lambda IDs must agree;
- entry handlers must exist;
- every edge must name a legal source outcome;
- edge input schemas must agree with destination lambda schemas;
- all handlers must be reachable from the entry;
- schemas must be acyclic;
- capabilities and effects must be declared and versioned;
- providers must declare coherent state, replay, and revocation semantics;
- tests must target existing lambdas and legal expected outcomes.

The reachability walk is deterministic because map keys are sorted before traversal. Diagnostics are also sorted by path, diagnostic ID, and message in `diagnostics.go`. Deterministic diagnostics matter operationally: activation systems and CI should not produce order-dependent output from Go map iteration.

The resulting `Program` is a finite, statically reviewable transition graph even though handler bodies remain dynamic. Static analysis can prove where control may resume and which effect families may be requested without interpreting arbitrary callback code.

8. The native module as a nominal type system

The native module in `internal/gojamodules/tinyidp/module.go` uses a particularly effective Goja technique: blank object handles branded by VM object identity.

When `A.lambda(id, spec)` is called, the loader:

1. validates the arguments;
2. extracts the callback as a `goja.Callable`;
3. records the callable in the collector under the stable ID;
4. creates a blank JavaScript object;
5. stores `collector.lambdas[object] = id`;
6. returns the blank object to JavaScript.

Workflow registration accepts only an object present in that identity map. An attacker cannot forge a lambda by constructing `{id: "signup.start"}` because properties are irrelevant. The nominal brand is the `*goja.Object` pointer known to Go.

The same mechanism is used for:

- host-defined field descriptors;
- host-defined action descriptors;
- invocation-scoped secret handles.

This gives an untyped language several nominal types without exposing a forgeable tag or a private symbol. It also makes misuse errors local and precise: a presentation field must be the exact object returned by the module, not an object with similar data.

8.1 Why blank objects are stronger than frozen tagged objects

A frozen tagged object such as `{kind: "lambda", id: "x"}` is immutable but forgeable. A JavaScript `Symbol` narrows accidental collisions but can still be copied if exposed. A Go-side identity table gives the host an unforgeable membership test for the lifetime of the runtime.

The pattern can be stated generally:

```
brand(v, T) := hostMap[T].contains(objectIdentity(v))
```

It is a useful bridge pattern whenever an embedded language needs opaque, nominal references to host-approved declarations.

9. A closed CommonJS world

`NewRuntimeFactory` disables implicit default modules, data-only default modules, and the ambient loader. The only registered native module is `tinyidp`.

Negative tests attempt to load `filesystem`, `process`, `execution`, `database`, `network`, `OS`, and arbitrary project modules. See `pkg/idpscript/invoke_test.go`.

This is a closed-world language profile:

```
JavaScript syntax and standard built-ins
+ require("tinyidp")
- filesystem
- process
- OS
- network
- database
- project module loader
```

The design correctly describes this as authority reduction, not a claim that hostile code is safely sandboxed in-process. The scripts are operator-trusted. The runtime profile constrains accidental and exploit-oriented authority but does not replace process isolation for adversarial code.

10. Deterministic callback registration

The core registration problem is subtle. A serializable program can contain the ID `signup.submitted`, but the executable closure exists only inside a specific Goja runtime. A worker pool therefore needs a reproducible linking rule:

```
Program callback ID <-> exact closure registered in this runtime
```

Tiny-IDP solves this by re-executing the same compiled source in every worker and checking several identities.

10.1 Canonical identities

`ComputeFingerprints` computes independent hashes for:

- source text;
- canonical program IR;
- sorted callback registry IDs;
- canonical schema registry.

The separate hashes answer different questions:

Fingerprint	What it detects
Source	Any executable source change, including callback bodies
Program	Any semantic registration/contract change
Callback registry	Missing, extra, or renamed VM-local closures
Schemas	Host/schema drift even when callback IDs are unchanged

10.2 Re-materialization checks

During compilation, `compiler.go` compiles the source, loads it into an isolated runtime, validates the resulting program, computes fingerprints, and stores an immutable artifact.

During worker loading, `runtime_factory.go`:

1. creates a fresh owned runtime;
2. installs a runtime-local collector;
3. executes the compiled program under the owner;
4. JSON-encodes `module.exports`;
5. obtains the collector's program independently;
6. requires canonical equality between the two;
7. validates the program again;
8. recomputes program, callback, and schema fingerprints;
9. compares them with the artifact;
10. requires a one-to-one callback-ID/lambda-spec set.

This is deterministic registration as an activation invariant, not as a coding convention.

10.3 What this does and does not prove

It proves that definition-time observable registration is reproducible across workers. It catches a program that conditionally registers different handlers based on time, random values, or runtime-dependent state if that difference reaches the program or callback registry.

It does not prove that callback behavior is deterministic for every input. A callback may intentionally depend on a declared capability, and a reused runtime may retain ordinary module state. Source identity and capability scoping make such behavior reviewable, but registration equality is not full semantic equivalence.

11. `module.exports` as an anti-ambiguity check

The runtime does not merely trust the collector. It also requires that `module.exports` be canonically equal to the value returned by `tinyidp.v1.program`. See [runtime_factory.go, lines 97-151](#).

This prevents several ambiguous module patterns:

- registering one program while exporting another;
- exporting an unrelated object after registration;
- relying on hidden collector side effects that are not visible in the module contract;
- accidentally omitting the program export.

The program has one authoritative public representation and one authoritative private callback registry, and activation checks that they correspond.

12. Immutable artifacts and defensive copies

Artifact stores:

- source text;
- a compiled `*goja.Program` reusable by independent runtimes;
- canonical program JSON;
- fingerprints.

`Artifact.Program()` decodes a fresh copy from canonical JSON. `RuntimeImage.Program()` similarly produces a deep copy. This removes a common embedded-runtime hazard: a caller cannot mutate the activation contract after validation by retaining a map or slice reference.

The artifact is therefore a reproducible recipe for runtime images, not a live runtime itself.

Part III - Request-time Goja execution

13. Single-owner runtimes

Goja runtimes are not treated as ordinary concurrent objects. Each worker owns one runtime image, and each invocation acquires the worker exclusively. Calls and asynchronous settlements are routed through the runtime owner's `Call` and `Post` operations.

`pkg/idpscript/pool.go` creates a bounded set of independently loaded workers. `Pool.Invoke` performs a lease:

```
acquire idle worker
-> increment active count
-> execute exactly one invocation
-> release if proven safe
-> otherwise close, discard, and load replacement
```

This is more than a mutex around `goja.Runtime`. The owner abstraction also serializes Promise settlement and interrupt cleanup with VM access. A request goroutine does not reach into a runtime while another goroutine settles a Promise.

13.1 Saturation is explicit backpressure

The pool is bounded. Acquisition waits on the idle-worker channel until the caller context ends. Failure is classified as `ErrRuntimeSaturated`, not as permission to create an unbounded runtime or run concurrently in an existing one.

This gives the runtime a clear capacity model:

```
maximum simultaneous JavaScript executions = worker count
```

The worker count is a security and operational budget as much as a performance setting.

14. The invocation context is copied, not shared

The boundary between Go values and JavaScript values is handled carefully in `invoke.go`.

The host does not pass decoded Go maps directly into Goja. It calls the runtime's own `JSON.parse` on the encoded input. The comment in `parseJSONValue` explains why: arrays and objects become ordinary guest JavaScript objects rather than reflective Go host objects. That distinction is important because:

- guest objects obey ordinary JavaScript property semantics;
- `Object.freeze` can make them read-only;
- scripts cannot retain a reference to a mutable Go map;
- encoding defines the complete crossing, including numeric and field behavior.

The runtime builds a context with these namespaces:

<code>ctx.input</code>	schema-validated public input
<code>ctx.cap</code>	invocation-scoped capabilities
<code>ctx.present</code>	data-only presentation builders
<code>ctx.challenge</code>	native challenge-request builders
<code>ctx.secret</code>	opaque request-scoped secret handles
<code>ctx.commit</code>	inert effect-plan builders
<code>ctx.evidence</code>	native-verified evidence projections

It then recursively freezes the context and all reachable guest objects. The test suite verifies both the top-level object and nested input/capability objects are frozen before a callback runs. See

`TestPoolInvokesSynchronousLambdaWithFrozenInput`.

Freezing does not make JavaScript pure. A closure may still mutate its own module variables. It does, however, prevent the callback from rewriting the host-projected invocation contract and confusing later native validation.

15. Invocation capabilities

A capability binding is deliberately small. `CapabilityBinding` contains:

- a stable capability ID and version;

- maximum input bytes;
- maximum output bytes;
- one native `Invoke(context.Context, json.RawMessage)` function.

A lambda receives a binding only if its `LambdaSpec` requires the exact ID and version. The host may supply other bindings to the call, but they are not inserted into `ctx.cap` unless declared by the lambda. Missing required bindings fail before the callback runs.

This produces an authority equation:

```
actual authority(lambda invocation)
  = declared requirements
    intersection host-supplied compatible bindings
```

There is no dynamic `getCapability(name)` operation. A script cannot enumerate a global service registry and cannot acquire authority by constructing a string.

15.1 Namespaced capability objects

Capability IDs such as `identity.lookup` are projected as nested objects:

```
await ctx.cap.identity.lookup(input)
```

The host constructs each namespace and rejects collisions. This provides readable JavaScript without changing the underlying stable ID used for validation and fingerprints.

15.2 Call budgets

Every capability call increments an atomic invocation counter before host work begins. A call above `MaxCapabilityCalls` throws a JavaScript `TypeError`. Input JSON is encoded and bounded before the native function is launched; output bytes are bounded before decoding and settlement.

The call budget limits both accidental loops and data-dependent amplification. A capability is not merely permission to access a service; it is permission to invoke a specific operation a bounded number of times in one lambda.

16. The Promise bridge

The capability bridge is one of the most technically interesting Goja sections. It coordinates JavaScript's single-threaded Promise semantics with concurrent native work.

When JavaScript calls a capability, `capabilityFunction`:

1. confirms the binding is still active;
2. validates argument count and call budget;
3. serializes and bounds the argument;
4. creates a Goja Promise, resolve function, and reject function on the owner thread;
5. increments the pending-settlement count;
6. launches native work under the invocation context;
7. catches any host panic and converts it to an error;
8. bounds and validates the native JSON result;
9. posts a settlement closure back to the runtime owner;
10. resolves with a guest value created through `JSON.parse`, or rejects with a stable generic reason;
11. waits until the posted settlement ran or the invocation context ended.

The native goroutine never invokes `resolve` or `reject` directly. It asks the owner to do so. This is the correct ownership discipline for an embedded single-threaded VM.

16.1 Structured settlement

`invocationBindings` tracks pending calls and an `errgroup`. After the lambda's own Promise resolves, the invocation still waits for all capability work to settle. This prevents a callback from starting host work, returning an outcome, and releasing the worker while a late goroutine still holds settlement functions associated with that runtime.

16.2 Temporal capability revocation

When the invocation ends, `bindings.close()` clears an atomic `active` flag and cancels the binding context. A capability function retained in a module global checks this flag before every use. The test fixture intentionally steals `ctx.cap.test.lookup` in one invocation and calls it in a later invocation; the retained function fails closed. See `TestInvocationCapabilityBudgetAndExpiredBindingFailClosed` and its source fixture in `invoke_test.go`.

This is a temporal object capability:

```
capability authority = object identity + active invocation epoch
```

A closure may retain the function object, but it cannot retain its authority.

16.3 Error redaction

Host panics and backend errors become a rejected Promise with a stable generic value such as `capability_failed`. This prevents arbitrary backend error text from entering script outputs, browser errors, or high-cardinality metrics. The tradeoff is reduced script-level diagnostics; operational detail must remain in native logs or audit channels.

17. Synchronous and asynchronous lambda results

A callback may return either:

- a normal JavaScript value; or
- a Goja Promise.

The worker invokes the closure under the runtime owner. A normal value is exported and JSON-encoded immediately. A Promise is retained as a VM-local object and polled for state through owner calls until fulfilled, rejected, or the invocation context expires. See `awaitPromise`.

This Promise is strictly an **in-request continuation**. It is never persisted and never leaves the worker. The HTTP request remains open, the worker remains leased, and all capability contexts remain bounded by the invocation deadline.

The implementation's 1 ms polling loop is pragmatic. It avoids cross-thread VM access, though an event-driven completion signal could reduce wakeups in a future implementation.

18. Interrupts and the worker safety proof

An invocation has a context deadline derived from the lambda budget. A `context.AfterFunc` interrupts the Goja VM if the context expires. The cleanup path calls `ClearInterrupt` through the runtime owner, but it still marks the worker unsafe if the interrupt callback may have run.

This conservative rule matters because interruption creates uncertain VM state:

- JavaScript may have been stopped between arbitrary operations;
- Promise jobs may remain queued;
- a host callback may have been in flight;
- module globals may be partially mutated;
- an interrupt flag may race with cleanup.

Rather than attempt to prove the heap reusable, the pool destroys it.

The invocation function returns a `safe` flag independently of the ordinary error. The pool releases only a safe worker. Any unsafe result triggers `discardAndReplace`, which closes the runtime image and loads a fresh one from the immutable artifact.

18.1 Fail-stop leasing

The lifecycle resembles a transactional resource lease:

```
begin lease
  precondition validation failure -> worker remains safe
  callback starts
    success + valid output + all work settled -> commit lease, reuse worker
    anything uncertain -> abort lease, destroy worker
```

The tests cover active infinite loops, caller cancellation, host panic, thrown exceptions, invalid output, timeout with a host operation that ignores cancellation, and late settlement. After each unsafe path, the discarded-worker and created-worker counters show replacement. See [invoke_test.go, lines 145-215](#).

18.2 Late settlement containment

A particularly strong test starts a capability that deliberately ignores cancellation, lets the lambda time out, replaces the worker, then allows the old capability to complete. The old settlement cannot reach the replacement runtime. This proves that runtime identity and invocation activity, not merely callback name, determine where a settlement may land.

19. Output decoding is a commit point

A JavaScript return value is not accepted merely because `json.Marshal` succeeded. `decodeOutcome` enforces:

- output length between 1 and the lambda's maximum;
- exactly one JSON value, with no trailing value;
- a recognized `Outcome` structure;
- an outcome kind declared by the lambda;
- continuation/presentation/challenge/effect fields legal for that kind;
- effect kinds allowed by the lambda;
- output `Value` conforming to the output schema.

Only after this validation and capability settlement does the worker become reusable.

This order prevents a malformed or oversized value from being treated as an ordinary application error. Invalid output indicates a breach of the runtime contract and causes worker disposal.

Part IV - Explicit durable continuations

20. A precise meaning of “serialized continuation”

The branch does not serialize a Goja continuation, stack, closure, Promise, or heap. It serializes a **defunctionalized continuation**.

A browser-spanning suspension is represented by:

```
next handler label
+ destination input schema
+ public carry environment
+ exact executable generation
+ native binding and replay metadata
+ presentation/challenge references
```

This is the classic shape produced when higher-order control is converted into a first-order state machine: a continuation function becomes a constructor tag plus the data needed by its eventual case handler.

In Tiny-IDP terms:

```
closure-like view:
  resume = submitted(capturedClient, capturedPublicValues, ...)

persisted view:
  ResumeHandlerID = "submitted"
  Carry           = {...public schema-checked values...}
  ProgramFingerprint = ...
  bindings        = ...
```

On the next HTTP request, the host resolves the exact generation, validates the record, projects new browser input, merges approved carry, and invokes the named handler as a fresh call.

The active design is explicit that browser waits do not leave a Promise pending. See [design-doc/03, sections 6.2-6.3](#).

21. Why VM serialization is rejected

Persisting a pending Promise or runtime heap would create several problems:

- the workflow would be tied to one runtime implementation and heap layout;
- process restart would require VM snapshot compatibility;
- the worker would remain occupied or the heap would need full serialization;
- closure environments could retain passwords, tokens, host functions, or stale capability objects;
- hot reload would have no principled rule for old closures under new source;
- state inspection, cleanup, and replay control would depend on opaque VM internals.

The explicit record is small, versioned, inspectable, schema-bounded, and independent of Goja.

`pkg/idpcontinuation/types.go` states the invariant directly: continuation values are pure Go data and must never contain Goja values, functions, Promises, or goroutine-local state.

22. The durable record as a typed security context

`WorkflowContinuation` contains more than a resume label. Its fields bind control state to the exact security context in which it was created:

- record version;
- keyed hash of the public handle;
- workflow ID and workflow version;
- resume handler ID and input schema;
- program fingerprint and schema version;
- authorization request digest;
- client ID, redirect URI, and client-generation identity;
- browser-binding hash;

- optional session and browser-context hashes;
- presentation state;
- public carry;
- native secret and evidence references;
- revision, creation time, expiration time, status, and terminal outcome.

This is not just persisted UI state. It is a resumption capability constrained by protocol, browser, client, generation, and type identity.

22.1 Public handles and stored hashes

A new continuation handle contains 32 random bytes encoded with unpadded URL-safe base64. The service stores only a domain-separated HMAC-SHA-256 of the raw handle. See [service.go, handle creation and hashing](#).

The raw browser capability is therefore not present in the row. A storage disclosure does not directly expose usable continuation URLs. Constant-time comparison is used for byte bindings.

22.2 Exact binding validation

A normal load requires complete expected bindings: workflow, client, redirect URI, client generation, request digest, and browser binding. Optional program/session/context expectations are checked if supplied. Mismatches are classified into bounded internal failure classes.

The public terminal code is intentionally uniform: `interaction_unavailable`. Internally, audit can distinguish missing, expired, replayed, revoked, browser mismatch, client mismatch, request mismatch, generation unavailable, generation mismatch, and invalid state. See [service.go, failure classification](#).

This separates operator diagnosis from browser enumeration resistance.

23. One-use transitions and linearizable replay control

The store interface is phrased in transitions, not CRUD. `pkg/idpcontinuation/store.go` exposes:

- `Create`;
- `Load`;
- `Advance`;
- `Consume`;
- `Revoke`;
- expired-record listing and deletion.

`Advance` must atomically mark the current record advanced and insert the next active record. It does not update the same record in place. The transition is:

```
Active(handle H, revision R)
-- Advance expected R -->
Advanced(H, R+1) + Active(new handle H2, revision 1)
```

`Consume` similarly changes an active record to a terminal consumed state under an expected revision. Concurrent requests race on the same revision; exactly one transition can win.

The shared store conformance suite includes a 24-way concurrent advance and requires one winner. The ticket records the same suite for memory and SQLite implementations, plus restart/resume without retaining a runtime.

23.1 Why replacement-chain advancement is useful

Creating a new handle at each browser boundary has several advantages:

- an observed old URL cannot be reused after successful advancement;
- each step has an independent expiration and presentation snapshot;
- the history distinguishes “advanced” from “consumed”;
- concurrent submissions have a natural compare-and-swap boundary;
- attachment cleanup can be associated with the record that owned the references.

24. Schema-checked carry and resume input

The continuation service resolves the exact program generation before validating a record. It checks:

- workflow ID and version;
- resume handler existence;
- handler-to-lambda mapping;
- lambda input schema equality with the record's pinned schema;
- public carry under that schema with sensitive fields forbidden.

On POST, `ValidateResumeInput` validates the ephemeral projected browser submission against the same destination schema. Sensitive values may exist in this ephemeral input only through separately projected handles; they are not permitted in durable public carry.

Advancement inherits immutable bindings from the current record and rejects any attempt to change workflow, program generation, client, request, browser, session, or browser context. See [service.go, validation and inheritance](#).

25. Generation pinning and semantic time travel

A long-lived browser interaction may cross a deployment. Resuming it on the newest script merely because that script is active would reinterpret old carry and control labels under new semantics.

Tiny-IDP treats a program generation as part of continuation identity. The signup executor's fingerprint joins source and program fingerprints, because a callback body can change while declared handlers and schemas remain identical. See `Executor.Fingerprint`.

`GenerationManager` maintains:

- one active warmed executor for new interactions;
- a map from retained fingerprints to executors;
- an ordered bounded retention set;
- atomic activation after compile, runtime loading, fingerprint checks, embedded tests, and warmup.

New interactions use `Active()`. Resumes must use `ExecutorFor(persistedFingerprint)`.

This creates explicit semantic time travel: old workflows continue under old executable meaning while new workflows use the new generation. If the required generation has been evicted, resume fails safely rather than guessing compatibility.

26. Cleanup is part of continuation semantics

Durable workflows may refer to native pending secrets or evidence. Expiration cleanup therefore calls an idempotent `AttachmentCleaner` before deleting the record. If attachment cleanup fails, the record remains so cleanup can retry. This avoids deleting the only durable pointer to an orphaned secret/challenge.

The order creates an intentional at-least-once cleanup contract:

```
clean referenced native state (idempotent)
then delete expired continuation
```

A process crash between the two may repeat attachment deletion, so idempotence is required by the interface.

Part V - Presentation, secrets, evidence, and native effects

27. Presentation is a typed protocol, not script-generated HTML

The browser boundary uses a second small language. `pkg/idpworkflow/descriptors.go` defines host-owned field and action descriptors.

A field fixes:

- stable field ID;
- HTML input name;
- label;
- value kind;
- normalization policy;
- requiredness;
- minimum and maximum lengths;
- sensitivity;
- autocomplete value;
- public redisplay policy.

An action fixes its stable ID, label, and whether it skips ordinary form validation. The native registry rejects duplicate IDs and duplicate input names.

JavaScript receives branded builder handles such as `A.field.email()` and `A.action.submit()`. It may select and order approved descriptors. It may not define arbitrary HTML, input names, form actions, hidden protocol fields, parser code, CSRF data, external URLs, or secret redisplay behavior.

This is a key interpreter design pattern: **presentation intent is a data language interpreted by the native renderer.**

28. Presentation validation connects UI to the workflow graph

`ValidatePresentation` proves several facts before persistence or rendering:

- the source handler exists and is allowed to return `present`;
- the requested resume handler is connected by a declared `present` edge;
- the edge's input schema equals the destination lambda's input schema;
- carry is public under that schema;
- the title and expiration are bounded;
- all fields/actions are registered and non-duplicated;
- at least one field and action are present;
- public values refer only to selected, non-sensitive, redisplayable fields and meet length bounds;
- field errors use selected fields and stable error categories.

The script cannot render a form that the static workflow graph does not know how to resume.

29. Exact POST projection

`ParseSubmission` accepts exactly the fields selected by the validated presentation. Before any lambda runs, native code rejects:

- unexpected parameters;
- duplicate singleton parameters;
- missing interaction, CSRF, continuation, or action values;
- actions not selected by the page;
- invalid UTF-8;
- invalid normalization;
- missing required values;
- out-of-bounds lengths;
- malformed normalized email addresses.

Public values are normalized and copied into a field-ID map. Sensitive values take a different path.

This prevents the common mistake of passing `r.Form` or a generic request map into a script. The browser input language is finite for each presentation.

30. Request-scoped secret handles

A sensitive field is first copied into a short-lived native byte slice. `SecretSet` associates it with a random token and returns a Go `SecretHandle` whose token field is unexported.

The Goja invocation layer converts each Go handle into a blank branded object. JavaScript can pass that object to a host-defined commit builder, but it cannot:

- read the secret bytes;
- read the token;
- JSON-serialize the handle into meaningful data;
- forge another valid handle;
- use the handle in another invocation.

The commit builder verifies object identity against the invocation's secret map and emits only the native token inside the internal effect payload. The trusted native committer later resolves that token through the same `Submission` and clones the bytes for immediate password work.

After commit or failure, `DestroySecrets` clears the byte slices and removes them from the map.

30.1 The authority path for a password

```
browser form value
-> bounded native byte slice
-> request-local SecretSet entry
-> branded Goja object
-> opaque handle reference in effect plan
-> native handle resolution
-> password policy/hash operation
-> clear cloned and original buffers
```

At no point does the JavaScript program receive a normal password string.

30.2 Handles are capabilities, not encryption

The handle does not cryptographically hide data from a host process. Its purpose is authority separation inside the interpreter boundary. It prevents the script language and ordinary serialization paths from gaining direct secret data.

31. `commit` as an algebraic effect request

A `commit` outcome contains an ordered list of `EffectPlan` values. Each plan names a stable effect kind and bounded JSON payload. The JavaScript callback does not receive a transaction or a store and cannot apply the plan.

This resembles an algebraic effect system:

```
script computation emits effect constructors
native handler interprets constructors under domain invariants
```

The analogy is especially strong because the effect set is declared in `LambdaSpec`, validated in the outcome codec, and interpreted by a domain-specific native function rather than a generic dispatcher.

32. Atomic signup commit

The checked-in `open_signup.js` is intentionally small. `signup.start` selects a native form. `signup.submitted` returns `ctx.commit.signup(...)` with public identity fields and opaque password handles.

The actual authority is `commitScriptedSignup`. It:

1. requires the exact effect sequence: create local identity, attach password credential, and optionally consume invitation;
2. strictly decodes each payload;
3. resolves password and confirmation handles from the current native submission;

4. verifies equality, non-emptiness, verified-email consistency, and registration policy;
5. prepares the account/credential operation natively;
6. generates a session handle natively;
7. opens one store transaction;
8. consumes the already loaded continuation through a transaction-scoped continuation store;
9. commits the prepared account and credential;
10. atomically redeems a durable invitation if present;
11. creates the browser session;
12. consumes the OAuth authorization interaction as approved.

This is the critical microkernel boundary. JavaScript decides that a declared workflow path wants signup. Go proves that the plan is valid and makes all identity-relevant state changes atomic.

32.1 ConsumeLoaded is a narrow transaction seam

The continuation service exposes `ConsumeLoaded` for native committers that already possess a binding-checked record. It accepts a transaction-scoped `Store`, but not a raw browser handle. This allows continuation consumption to join the account transaction without widening the public resume API.

That small API detail prevents the committer from becoming a second general continuation endpoint.

33. Native challenges and unforgeable evidence

Email verification exercises a second durable machine. `pkg/idpemailchallenge` owns pending challenge state and verified evidence.

JavaScript may return a typed `challenge` request naming:

- challenge kind;
- email and approved template;
- resume handler;
- maximum attempts/resends;
- continuation carry and expiration.

Native code then:

- validates the declared challenge edge;
- generates the code;
- stores only a domain-separated keyed code hash;
- binds the record to workflow, handler, program generation, client, client generation, browser, and expiry;
- sends through a narrow `Mailer` interface that accepts an approved template request, not an SMTP client;
- advances the workflow continuation to a native email-code page.

`VerifiedEmailEvidence` is produced only by a successful native verification transition. The resumed lambda receives a JSON projection of that evidence, not the code and not a script-created `verified: true` marker.

33.1 References versus evidence

The architecture distinguishes three values:

1. **Challenge reference:** safe durable pointer, no code/hash.
2. **Pending/verified native record:** authoritative store state, never script-visible.
3. **Evidence projection:** immutable bounded fact produced after native verification.

A continuation may retain the reference. On later steps, Go rehydrates evidence by loading the native record and rechecking bindings. For replay-sensitive terminal effects, a separate `ConsumeEvidence` transition makes evidence one-use.

This is evidence-carrying control flow: later handlers may branch on a fact, but only the native verifier can create the fact.

34. Multi-request workflow execution

The production adapter's resume path demonstrates the entire interpreter chain:

```
POST /authorize
-> load OAuth interaction
```

```
-> load and bind-check workflow continuation
-> route to persisted program generation
-> reconstruct host field/action descriptors
-> parse exact form and create secret handles
-> verify or rehydrate native challenge evidence
-> build destination-schema input
-> invoke persisted handler ID
-> interpret present/challenge/commit
-> atomically advance or consume state
```

The browser never chooses the handler ID. It submits an opaque continuation handle. The handler comes from the binding-checked record. See `resumeScriptedSignup`.

This avoids a frequent workflow vulnerability: accepting a step name or next-state field directly from the browser.

Part VI - Providers and policy interpreters

35. Providers are typed callback collections

`pkg/idpprogram/providers.go` defines provider kinds for identity, invitation, authorization, claims, and presentation. Each provider declares:

- stable provider ID and version;
- kind;
- state model;
- replay-protection semantics;
- revocation semantics;
- named handler bindings.

The state/replay/revocation fields force an important question into the contract: what makes a provider result trustworthy over time?

Examples include:

- virtual providers with no durable row;
- signed stateless invitations whose replay bound is expiry and whose revocation is key rollover;
- durable one-time invitations whose replay protection is atomic consumption and whose revocation is a store transition.

A provider cannot merely label itself “one time”. Program validation requires a coherent combination of state and replay semantics that the native host knows how to support.

36. Provider invocation reuses the same kernel

`ProviderInvoker` resolves a provider and stable handler through the immutable program contract, then delegates to the same pool used by workflow lambdas.

Provider callbacks therefore inherit:

- exact schema validation;
- declared capabilities;
- timeout/call/output budgets;
- Promise ownership;
- frozen inputs;
- outcome validation;
- unsafe-worker disposal.

This avoids creating a looser second scripting engine for providers.

37. Authorization policy remains downstream of native proof

`pkg/idppolicy/executor.go` binds authorization, claims, and presentation callbacks to the Goja runtime but exposes no HTTP, Fosite, cookie, credential, key, session, or store authority.

For authorization:

- native code first constructs an immutable, cloned authorization input after protocol validation;
- the selected provider may return `complete`, `deny`, `skip`, or `error` under its contract;
- unsupported workflow outcomes fail closed;
- the native `NormalizeAuthorizationDecision` validates the result before the protocol continues.

The script can influence policy but cannot manufacture the native proofs required to issue code or tokens.

38. Claims are additive under native protection

The claims callback receives a cloned native view and may return only an additional claim map. The native layer validates that output against the base claims and preserves protocol-owned names such as issuer, subject, audience, expiration, nonce, and authentication time.

This is a constrained merge, not arbitrary token mutation. The trusted token constructor remains in Go.

39. Presentation policy is decoration-only

The presentation provider receives a validated input and may return a bounded output such as a title. It cannot alter form controls, hidden protocol values, security headers, or flow state. This demonstrates a useful principle for extensible identity systems:

Separate cosmetic policy from control-flow and protocol policy, and give each a different output type.

Part VII - Verification as a separate language

40. Production scripting and verification scripting are different interpreters

The repository contains a second Goja compiler in `internal/gojaverify/compiler.go`.

It runs JavaScript in an isolated runtime with only `tinyidp/verify`. The script produces a data-only `verifyplan.Plan`. It receives no provider, store, network, filesystem, clock, assertion implementation, or production capability.

Native Go later executes the plan through a driver and assertion registry.

This separation prevents test tooling from becoming a production backdoor. A verification script can request a registered scenario action in an offline harness; it cannot call the production policy capability registry or mutate a live authorization decision.

41. Step registries make scenario languages finite

Originally, a verification step was a string kind plus arbitrary JSON. The branch head adds `StepRegistry`: an explicit map from step kind to exact native parameter validator.

`Plan.ValidateWithSteps` requires a non-empty registry and validates every step before the driver sees the plan. Unknown kinds, malformed JSON, unknown fields, non-object values, trailing values, and domain-specific invalid parameters fail during materialization.

This is another interpreter lesson:

```
string opcode + arbitrary JSON = an open execution language
registered opcode + exact codec = a finite reviewable language
```

The commit message at the analyzed branch head explicitly frames the change this way.

42. Assurance vocabulary as an inter-interpreter ABI

`internal/assurance/vocabulary.go` defines versioned stable IDs for:

- resources;
- facts;
- obligations;
- handlers and schemas;
- capabilities and effects;
- evidence and diagnostics;
- observations and outcomes;
- steps and properties.

The package imports no protocol, persistence, Goja, or HTTP package. It is intended as a dependency-neutral vocabulary shared by configuration, native transition descriptions, verification scenarios, traces, static analysis, and formal models.

`internal/assurance/schemas.go` then separates three schemas:

1. **Configuration reference:** what compiled program generation is desired.
2. **Native transition catalog:** what host-owned transitions read, write, require, produce, discharge, effect, and observe.
3. **Scenario/trace records:** what a test requests and what runtime instrumentation actually observed.

The separation is crucial. Configuration metadata cannot claim a transition ran, and a transition descriptor cannot fabricate a trace event merely because it says one should occur.

At the analyzed commit, stable vocabulary, the three-schema boundary, obligation codecs, and registered verification-step codecs are complete. Full event-to-transition mapping, proof objects, generated analyzer/model metadata, counterexample replay, and the final cross-phase completion gate remain open in the task ledger.

Part VIII - A formal reading of the design

43. Defunctionalization and continuation-passing style

The most accurate theoretical description of browser continuation handling is **defunctionalized continuation-passing style**.

At the JavaScript API level, `ctx.present.form({resume: "submitted", carry: ...})` explicitly supplies the continuation label. The native module returns an outcome containing that label and environment. The continuation service persists them. On the next request, the workflow interpreter performs a case analysis on the label by resolving the corresponding handler.

```
higher-order intuition:
  suspend(form, input => submitted(carry, input))

first-order representation:
  Present {
    ResumeHandlerID: "submitted",
    Carry: carry
  }

resume interpreter:
  switch ResumeHandlerID {
    case "submitted": invoke callbackRegistry["signup.submitted"]
  }
```

The representation is deliberately not a complete language stack. It serializes only an approved control label and approved environment. This greatly narrows the state that must survive restart.

44. Algebraic effects without a general effect handler

`OutcomeCommit` plus `EffectPlan` resembles algebraic effects, but the implementation is intentionally domain-specific.

A general algebraic-effect runtime might let arbitrary handlers interpret arbitrary operations. Tiny-IDP does not. The effect interpreter is a named native function such as `commitScriptedSignup`, and it accepts only an exact effect sequence. That restriction is valuable in identity code because atomicity and authority are domain properties, not generic middleware properties.

The practical effect typing is:

handler declaration	permits effect kinds
outcome validator	rejects undeclared kinds
native domain committer	rejects wrong order/shape/semantics
transaction	applies approved effects atomically

Each layer narrows the previous one.

45. Object capabilities and temporal authority

The system uses object-capability ideas at two levels.

45.1 VM-local nominal references

Branded Goja objects grant the ability to refer to a host-approved lambda, field, action, or secret. Possession is checked by object identity.

45.2 Invocation capabilities

Functions in `ctx.cap` grant authority to request one exact native service operation. The authority is:

- explicitly inserted;
- non-discoverable outside the declared namespace;
- versioned;
- input/output bounded;
- call-count bounded;
- deadline-bound;

- revoked when the invocation ends.

This temporal revocation is stronger than merely deleting `ctx.cap` after the call. A retained function checks the invocation epoch and remains inert.

46. Staging and partial evaluation

The top-level JavaScript execution is a staging phase. It performs author-friendly computation to construct a static program description and callback registry. The host then validates and canonicalizes the static portion before request traffic.

The split resembles partial evaluation:

- definition-time choices become IR maps, IDs, schemas, and edges;
- runtime-dependent choices remain callback bodies and declared capabilities;
- protocol and storage transitions remain native.

Staging reduces the dynamic surface. A typo in a handler ID, illegal edge, unknown schema, undeclared effect, provider replay mismatch, or callback registry drift is rejected at activation rather than on a browser request.

47. Linearizability at the interpreter boundary

The interpreter does not stop at language-level safety. Durable state transitions are specified as one-use atomic operations. Revision-checked `Advance` and `Consume`, one-time challenge verification, invitation redemption, and the signup transaction are all designed around a single winner.

This is important because a browser workflow interpreter is concurrent even if each Goja runtime is single-threaded. Multiple HTTP requests can present the same handle simultaneously. The correctness property must therefore be stated over the store transition, not over the JavaScript callback.

The shared continuation conformance suite and the repository's Porcupine/Rapid assurance work are evidence that concurrency semantics are treated as part of the language implementation.

48. Nominal IDs as an ABI

Stable IDs act as an application binary interface among several interpreters:

```
JavaScript source
-> Program IR
-> callback registry
-> continuation rows
-> provider routing
-> audit/metrics dimensions
-> verification plans
-> assurance transition catalog
-> model/static-analysis artifacts
```

An ID is not just a display name. It is a durable reference whose version and meaning may outlive a process and cross tooling boundaries. This explains the strict identifier grammar, explicit versions, canonical sorting, and rejection of unknown future values.

Part IX - Security invariants

49. Core invariants and their enforcement points

Invariant	Primary enforcement
No Goja value enters the durable program	pkg/idpprogram has no Goja types; compiler crosses through JSON
No Goja heap/Promise is a browser continuation	WorkflowContinuation is pure Go data; fresh invocation on resume
Every executable callback has a stable declaration	lambda callback ID plus per-runtime collector registry
Every worker links the same program and callbacks	program/callback/schema fingerprints checked during load
A script cannot forge a declaration handle	Go map keyed by *goja.Object identity
A lambda receives no ambient host services	closed module registry and explicit invocation capabilities
A capability cannot outlive its invocation	atomic active flag, canceled context, owner-scoped settlement
VM access is single-owner	exclusive worker lease plus owner Call/Post
Uncertain VM state is never reused	unsafe flag, close, discard, replacement
Inputs and outputs are bounded data	schema validation, JSON crossing, byte limits, single-value decoder
Durable carry cannot contain sensitive fields	ValidatePublicJSON under destination schema
Browser cannot choose an arbitrary handler	handler ID comes from the binding-checked continuation
Browser cannot submit extra workflow fields	exact native field/action projection and singleton parsing
Secrets are not normal JavaScript strings	native byte buffers plus identity-branded opaque handles
Script cannot directly mutate identity state	inert effect plans plus native committers
Signup state changes are all-or-nothing	one native transaction consumes continuation and interaction, writes account/credential/session/invite
Challenge evidence cannot be forged by script	native hash verification creates evidence; references are rechecked
Continuations cannot silently change semantics after reload	source+program fingerprint pinning and retained generations
Replay has one winner	revision CAS, atomic advance/consume, one-use challenge/store operations
Public errors do not enumerate internal state	uniform browser code plus bounded internal failure class
Verification scripts do not gain production authority	separate module/runtime profile and data-only plan compilation
Scenario execution language is finite	non-empty StepRegistry with exact parameter codecs

50. Authority matrix

Concern	JavaScript may do	JavaScript may not do	Native owner
Workflow	choose declared outcome/edge	create undeclared handler or resume arbitrary state	program validator and workflow executor
Async lookup	call declared bounded capability	reach network/store directly or reuse capability later	capability binding
Browser UI	select registered fields/actions and safe public values	emit HTML, form URL, hidden protocol fields, headers, cookies	renderer and request parser
Password	pass opaque handle to commit builder	read, log, serialize, retain, hash, or compare plaintext	submission secret set and account service
Email proof	request native challenge and branch on evidence	generate/verify codes or assert verified state	challenge service/store/mailler
Persistence	return effect plan	open transaction or invoke CRUD	named native committer
OAuth/OIDC	influence bounded policy decisions	mutate Fosite request, issue code/token, set protected claims	Fosite adapter and policy seam

Concern	JavaScript may do	JavaScript may not do	Native owner
Reload	define a new source generation	reinterpret old continuation under new code	generation manager
Tests	author data-only verification plans	call live production capabilities	verify compiler, registry, driver

Part X - Critical evaluation

51. What is unusually strong

51.1 The design refuses false unification

Many extensibility systems expose one generic plugin context and one generic result. Tiny-IDP keeps separate languages for:

- program definition;
- workflow outcomes;
- presentation intent;
- invocation capabilities;
- native effect plans;
- durable continuations;
- provider contracts;
- verification scenarios;
- assurance observations.

This creates more types and adapters, but it preserves the different trust and lifetime semantics of each boundary.

51.2 Durable control is explicit at the API level

The system does not hide browser suspension behind a magical long-lived `await`. The author can see every durable boundary in the handler graph. That makes restart behavior, secret lifetime, generation compatibility, and replay state reviewable.

51.3 Determinism is checked by re-materialization

The runtime does not assume that named callbacks were registered consistently. It executes the compiled source in each worker and compares independent representations. This is a rigorous answer to a common embedded-language problem: the program graph is durable, but functions are heap-local.

51.4 Capability lifetime is enforced inside retained closures

It is easy to delete a context object after a call; it is harder to prevent a callback from retaining one of its functions. The active flag and late-settlement design address the actual closure-retention problem.

51.5 Unsafe runtimes are treated as contaminated

The conservative discard policy avoids reasoning from incomplete facts. Rather than deciding that a timeout probably left the VM usable, the pool makes reuse contingent on a successful proof: valid outcome, no interruption uncertainty, and all async work settled.

51.6 The commit boundary is named and domain-specific

`commitScriptedSignup` is not a generic effect executor. It revalidates the exact plan and joins all security-relevant writes in one transaction. That preserves auditability and supports targeted static analysis.

51.7 Verification is not smuggled into production

The separate `tinyidp/verify` module and step registry recognize that test expressiveness has a different authority model. This is a strong defense against “debug hooks” becoming production mutation channels.

52. Residual risks and design tensions

The following points are analytical observations, not claims that the branch is defective. They identify where the architecture still relies on policy, operational discipline, or future assurance work.

52.1 This is not an adversarial-code sandbox

The design explicitly treats scripts as trusted operator code. Module isolation, budgets, and worker disposal reduce authority and contain many failures, but an in-process JavaScript engine is still part of the serving process. Hostile multi-tenant code would require stronger isolation such as a separate process or another security boundary.

52.2 Successful workers retain ordinary JavaScript state

A worker loads the module once and is reused after safe invocations. The test suite proves that retained capability functions lose authority, but ordinary module variables can still persist and influence later calls. That can be useful for pure caches, but it can also introduce request-history dependence.

A production profile could strengthen this area by:

- documenting that callbacks must be observationally stateless except for immutable definitions;
- adding static lint rules for top-level mutable bindings;
- exposing only host-owned bounded caches as capabilities;
- optionally using fresh runtimes for especially sensitive callback classes.

52.3 Registration determinism is not behavioral determinism

Fingerprints catch differences in source, program, callbacks, and schemas. They do not prove that a callback returns the same result for the same input. Declared capabilities are expected sources of nondeterminism. Standard JavaScript time/random facilities, if available in the selected runtime profile, could also affect behavior without changing registration.

The right target is not universal determinism; it is **declared nondeterminism**. A future profile could restrict direct time/random use and require clock/random capabilities so tests and traces can account for them.

52.4 Property enumeration order should not become an API

Capability namespaces are assembled from Go maps. The code validates by stable IDs, but a script that branches on `Object.keys(ctx.cap)` order could observe implementation-dependent insertion order. Documentation and tests should treat capability access by name as the supported interface and avoid promising enumeration order.

52.5 Promise polling is simple but not free

The 1 ms Promise-state polling loop is safe under the owner discipline, but many long-running in-request capabilities could create wakeup overhead. An owner-posted completion signal could preserve semantics with less polling if profiling justifies it.

52.6 Error redaction shifts burden to native observability

Generic capability rejection and generic browser continuation errors avoid leakage, but they reduce diagnosis from JavaScript and the browser. The native audit/metrics path must therefore retain enough bounded information to distinguish backend failure classes without exposing secret or attacker-controlled text.

52.7 Bounded generation retention must match continuation lifetime

The manager retains a configured number of previous generations, while continuations have time-based expiration. If deployments outpace retention, a still-unexpired continuation can become unresumable. This is fail-closed and semantically correct, but product policy should align retention, deployment frequency, and maximum continuation TTL.

An alternative is reference-counted retention by live continuation generation, though that adds storage coordination and cleanup complexity.

52.8 Handle-key rotation requires an explicit operational story

Continuation handles and challenge codes use keyed domain-separated hashes. Rotating those keys can invalidate live records unless the service supports key versions or a grace keyring. The record version exists, but the examined continuation record does not visibly carry a hash-key ID. Operational documentation should state whether rotation intentionally expires all live workflows or supports overlap.

52.9 Source hashing is intentionally conservative

The executable generation fingerprint includes the source hash. A comment-only change therefore creates a new generation even if program and callback behavior are unchanged. This is conservative and reproducible, but may increase retained generations. A more semantic body hash would be complex and easier to get wrong; the present choice is defensible.

52.10 Native effect catalogs remain intentionally manual

Adding a new effect requires Go code, validation, atomicity design, and tests. This is friction, but it is security-positive friction. A generated generic dispatcher would risk turning the effect plan into a dynamic transaction language.

52.11 Some assurance work is still in progress

The task ledger leaves cross-phase stable-ID/trace/model properties and the overall completion gate unchecked. Phase 9 still lacks complete event-to-transition mapping, unexported authorization proof objects, generated analyzer/model metadata, normalized counterexample replay, and selected transition-kernel refactoring.

The implementation should therefore be described as a substantial completed lambda-first runtime with an active assurance-consolidation program, not as a finished formal verification system.

52.12 Mail delivery and durable challenge creation are not one transaction

The challenge service stores the pending record before calling the mailer. A delivery failure can leave a record that was not successfully delivered. The surrounding flow maps failure safely, and cleanup/retry semantics can address stale state, but exactly-once delivery is not implied. This is a normal distributed-systems boundary and should remain explicit.

53. Why the design is rigorous without pretending JavaScript is safe by itself

The branch does not rely on one protection. It composes independent checks:

```
closed module world
+ pure IR
+ deterministic materialization
+ nominal handle branding
+ frozen copied inputs
+ explicit capabilities
+ resource budgets
+ closed outcome algebra
+ worker contamination policy
+ durable typed continuations
+ one-use storage transitions
+ native evidence
+ native atomic commit
+ generation pinning
+ separate verification language
```

No single item proves the system safe. The strength comes from the absence of a bypass path that silently changes trust domains.

Part XI - Testing and operations as interpreter design

54. Tests target language-boundary failure modes

The `pkg/idpscript` tests are notable because they do not stop at happy-path callback execution. They exercise:

- independent runtime images with identical callback registries;
- runtime schema drift;
- forged lambda handles;
- missing program export;
- unbounded definition-time loops;
- forbidden module families;
- frozen guest input;
- Promise-returning capabilities;
- capability call budget exhaustion;
- retained expired capability functions;
- capability host panic;
- missing and undeclared capabilities;
- caller cancellation;
- active JavaScript deadline interruption;
- late host settlement after worker replacement;
- thrown exceptions;
- malformed output;
- pool saturation;
- concurrent exclusive invocations under race testing.

These are interpreter tests, not only business-flow tests. They probe ownership, lifetime, linking, serialization, and contamination boundaries.

55. Shared store conformance

The continuation store uses one conformance suite for memory and SQLite implementations. This is important because the service semantics depend on atomic one-use transitions, not merely method signatures. A backend is conformant only if concurrent advance/consume, replay classification, expiry, revocation, cleanup retry, and restart behavior match.

A similar pattern appears in challenge and invitation stores. The interpreter's durable semantics are therefore portable across storage backends only through behavioral conformance.

56. Embedded script tests and activation

Programs may register declarative tests. The signup executor runs them with a deliberately small set of deterministic fake capabilities: clock, random, mailer, identity lookup, invitation lookup, and store get. The fake is available only if the lambda declared the matching capability.

The generation manager runs embedded tests while warming the candidate. Compile, validation, binding, fingerprint, test, warmup, or readiness failure leaves the current generation active.

This is transactional deployment:

```
build candidate
-> validate
-> materialize every worker
-> verify linkage
-> run deterministic tests
-> verify readiness
-> atomic publish
-> retain/drain prior generations
```

57. Bounded observability

The runtime exposes bounded counters for:

- workers created/discarded;
- pool capacity, active, and idle workers;
- invocation counts, failures, interruptions, outcomes, and latency;
- continuation create/load/failure/replay/expiry/cleanup;
- generation activation/failure/retention/eviction;
- audit failures.

Audit events use stable result/reason categories and source/program fingerprints. They intentionally omit JavaScript source, raw exceptions, passwords, invite codes, emails, and unbounded callback labels.

Observability is treated as another output codec: dimensions must be stable and low-cardinality.

Part XII - Transferable engineering patterns

58. Pattern catalog

Pattern 1: Compile dynamic configuration into a pure IR

Keep VM values and host objects out of the durable contract. Make the IR independently validatable and serializable.

Pattern 2: Defunctionalize durable callbacks

Persist a stable callback ID plus a bounded environment. Rehydrate the exact code generation and invoke fresh.

Pattern 3: Link by deterministic re-registration

Execute the same compiled source in each runtime. Compare program, callback, and schema identities before accepting a worker.

Pattern 4: Use host-side object identity for nominal brands

Return blank objects and remember them in VM-local host maps. Accept only identities created by the current module/runtime.

Pattern 5: Separate capability calls from effect commits

Capabilities perform bounded observations or services during one invocation. Effects are inert requests interpreted by a native transaction boundary.

Pattern 6: Make capability lifetime explicit

A capability should carry an invocation epoch or active flag so a retained closure cannot reuse authority later.

Pattern 7: Copy through the guest's data codec

Use JSON or another explicit codec to create guest-native values. Avoid exposing reflective host maps and pointers unless that is the intended API.

Pattern 8: Freeze the projected contract

Recursive freezing is not a sandbox, but it prevents callbacks from rewriting host-projected inputs and capability namespaces.

Pattern 9: Lease runtimes transactionally

Return a worker to the pool only after a positive safety proof. Destroy uncertain runtimes.

Pattern 10: Treat browser resumption as a one-use store transition

Use high-entropy public handles, stored keyed hashes, exact bindings, revision checks, and atomic advance/consume.

Pattern 11: Pin durable state to executable semantics

Persist source/program generation identity. Do not resume old state under whichever code is currently active.

Pattern 12: Interpret UI intent natively

Let scripts select registered fields/actions; keep parsing, rendering, CSRF, origins, headers, and secret handling native.

Pattern 13: Represent verified facts as native evidence

Scripts may consume an immutable evidence projection but cannot construct the authoritative fact.

Pattern 14: Use explicit finite registries for test languages

A string opcode with arbitrary JSON is a language. Give it a registry, exact codecs, versions, and pre-execution materialization.

Pattern 15: Share IDs across execution and assurance

Stable resource/fact/effect/step/observation/property IDs reduce drift among runtime code, tests, traces, analyzers, and models.

59. Review checklist for an embedded policy interpreter

A reviewer can apply the following questions to another system:

1. Can any VM value reach persistence or survive runtime replacement?
2. Are callbacks referenced by stable IDs, and is every runtime's registry verified?
3. Can definition-time code access ambient filesystem, process, network, or project modules?
4. Does a callback receive only declared, versioned capabilities?
5. Can a retained closure use a capability after the invocation ends?
6. Are input, output, call count, time, and worker count bounded?
7. Are guest values copied or are mutable host maps/pointers exposed?
8. Is output a closed algebra or an open object convention?
9. Does malformed output contaminate or merely return to the pool?
10. Are browser continuation labels accepted from the browser or loaded from trusted state?
11. Is continuation carry schema-checked and secret-free?
12. Are replay and concurrent submission resolved atomically in storage?
13. Is resumption pinned to an exact executable generation?
14. Can scripts render HTML or parse arbitrary requests?
15. Can scripts read raw passwords/codes or only use opaque handles/evidence?
16. Can scripts open transactions or only return inert effect requests?
17. Is the final identity/protocol commit a named native atomic operation?
18. Are verification/test capabilities physically separate from production capabilities?
19. Are diagnostics, audit, and metrics bounded and redacted?
20. Which invariants are implemented, which are tested, and which remain design intent?

Part XIII - Worked execution trace

60. Open signup from source to commit

This trace follows the checked-in open-signup program.

60.1 Definition time

The source requires the only native module and defines two named lambdas:

```
signup.start
  input: signupStartInput
  outcomes: present
  capabilities: none
  effects: none

signup.submitted
  input: signupSubmittedInput
  outcomes: commit
  capabilities: none
  effects: createLocalIdentity, attachPasswordCredential
```

The program registers workflow `signup`, version 1, with entry `start` and one `present` edge from `start` to `submitted`.

The compiler produces:

- canonical Program JSON;
- callback IDs `signup.start` and `signup.submitted`;
- source/program/callback/schema fingerprints;
- compiled Goja program.

60.2 Worker activation

Each worker executes the compiled module in a fresh owned runtime. The native collector records two VM-local closures and two branded lambda handles. Exported and collected programs must agree. Fingerprints must match the artifact.

60.3 Initial request

Native OAuth code validates client, redirect URI, request, and interaction. It calls `executor.Start` with a redacted immutable input containing only client ID, redirect URI, requested scope, interaction ID, and whether a browser session exists.

The pool invokes `signup.start`. The callback selects registered display-name, email, password, and password-confirmation fields plus submit/deny actions. It returns a data-only presentation outcome with resume label `submitted`.

Native code validates the presentation against the graph and registry, creates a durable continuation bound to the OAuth request/browser/generation, and renders the native page.

60.4 Browser POST

The browser submits the interaction handle, CSRF token, workflow continuation handle, selected action, and selected fields.

Native code:

- loads and binding-checks the continuation;
- routes to the persisted executor generation;
- reconstructs the exact field/action descriptors;
- rejects extra or duplicate form fields;
- normalizes public fields;
- stores password values in a request-local `SecretSet`;
- creates opaque password handles;
- validates destination input against `signupSubmittedInput`.

60.5 Submitted handler

The pool invokes `signup.submitted` with:

```
ctx.input.displayName ordinary frozen string
ctx.input.email        ordinary frozen string
ctx.secret.password    branded opaque object
ctx.secret.passwordConfirmation branded opaque object
```

The callback cannot inspect the passwords. `ctx.commit.signup` verifies the secret object identities and returns an effect plan containing public identity data and native secret tokens.

60.6 Native commit

The adapter requires the exact effect sequence, resolves the secret tokens against the current submission, verifies password confirmation and policies, prepares the account, and opens one transaction.

Inside the transaction it:

- consumes the workflow continuation;
- commits identity and password credential;
- creates the browser session;
- consumes the authorization interaction as approved.

Only after the transaction succeeds does the adapter set the session cookie and continue consent or authorization-code issuance.

60.7 Restart property

Between the initial page and the POST, the process may restart. The continuation contains no Goja object. A new service instance can load the record, resolve the retained program generation, reconstruct a fresh invocation context, and invoke `signup.submitted` in a different runtime.

Part XIV - Goja construct index

61. Goja APIs and their roles

Construct	Use in Tiny-IDP	Design significance
goja.Compile / *goja.Program	Compile once; execute in independent runtimes	Separates source compilation from runtime ownership
goja.New	Create isolated definition/worker runtimes	No shared heap across workers
goja.Callable / AssertFunction	Store named callback closures in collector	VM-local executable registry behind stable IDs
*goja.Object identity	Brand lambdas, fields, actions, secrets	Unforgeable nominal references within one VM
Runtime.NewPromise	Create JS Promise for native capability call	Async bridge created on owner thread
Promise resolve/reject callables	Settle native result	Posted back to owner; never called from host goroutine directly
Runtime.Interrupt	Stop deadline-exceeding JavaScript	Bounded execution with conservative worker disposal
Runtime.ClearInterrupt	Clean owner state after interrupt path	Ordered cleanup; uncertainty still marks worker unsafe
Runtime.NewTypeError	Fail invalid invocation/capability calls	Stable script-visible contract error
Value.Export()	Cross result/module export into Go	Immediately re-encoded and validated; not persisted as VM value
guest JSON.parse	Create ordinary JS input/result objects	Avoid reflective Go host objects
guest Object.freeze	Freeze context recursively	Prevent mutation of host-projected invocation contract
CommonJS module.exports	Single explicit program export	Checked against independently collected Program
require.Registry	Closed module resolver	Only approved native modules are resolvable
owner Call	Execute all VM-touching operations	Single-thread ownership
owner Post	Schedule Promise settlement	Safe cross-goroutine handoff

62. Host-language constructs that complete the interpreter

The Goja pieces work because they are embedded in complementary Go mechanisms:

- contexts and deadlines;
- `context.AfterFunc` for interruption;
- atomic flags/counters for capability epochs and metrics;
- `errgroup` for async settlement;
- bounded channels for worker leasing;
- canonical JSON and SHA-256 fingerprints;
- HMAC for opaque handle/code lookup;
- transaction-scoped interfaces;
- immutable copies through JSON;
- typed error classes and stable diagnostics.

The interpreter is therefore not “inside Goja.” It is distributed across language runtime, pure IR, storage protocol, and native domain code.

Part XV - Source map

63. Recommended reading order

1. `pkg/idpprogram/program.go` - pure program/workflow representation.
2. `pkg/idpprogram/lambda.go` and `outcomes.go` - type/effect/outcome contract.
3. `pkg/idpprogram/validate.go` and `value.go` - static and value validation.
4. `internal/gojamodules/tinyidp/module.go` - collector, branded handles, builders.
5. `pkg/idpscript/compiler.go`, `artifact.go`, and `runtime_factory.go` - staging and deterministic linking.
6. `pkg/idpscript/pool.go`, `invoke.go`, and `capabilities.go` - runtime ownership, Promises, and capability lifetimes.
7. `pkg/idpcontinuation/types.go`, `store.go`, and `service.go` - serialized continuation semantics.
8. `pkg/idpworkflow/descriptors.go`, `presentation.go`, `submission.go`, and `secrets.go` - typed browser and secret boundary.
9. `pkg/idpsignup/open_signup.js`, `executor.go`, and `internal/fositeadapter/scripted_signup.go` - end-to-end vertical slice and atomic effect commit.
10. `pkg/idpemailchallenge` - native restart-safe challenge evidence.
11. `pkg/idpprogram/providers.go`, `pkg/idpscript/providers.go`, and `pkg/idppolicy/executor.go` - provider/policy reuse of the kernel.
12. `internal/gojaverify/compiler.go`, `pkg/verifyplan/plan.go`, and `registry.go` - separate verification interpreter.
13. `internal/assurance` - shared assurance ABI.
14. `design-doc/03` - normative rationale.
15. `tasks.md` and `changelog.md` - implementation sequence and remaining work.

Conclusion

Tiny-IDP's Goja work is best viewed as a disciplined interpreter construction project rather than a scripting feature.

The design accepts that JavaScript closures are useful for local policy and orchestration, but rejects the idea that they should become durable state, ambient authority, protocol proof, or transaction ownership. It replaces those unsafe equivalences with explicit representations:

- callbacks become stable IDs linked to VM-local closures;
- runtime contracts become pure IR;
- authority becomes versioned invocation capabilities;
- asynchronous native work becomes owner-settled bounded Promises;
- browser waits become defunctionalized continuation records;
- secrets become request-scoped opaque handles;
- verified facts become native evidence;
- state mutation becomes inert effect plans interpreted by named native committers;
- reload becomes generation-pinned activation;
- tests become data-only plans materialized against finite native registries.

The result is not a generic workflow engine and not a JavaScript identity provider. It is a Go identity microkernel with several deliberately small languages around it. That refusal to collapse distinct trust domains is the branch's most important interpreter contribution.